

Dil Seçeneği Ekleme

Her şeyden önce i18n dosyalarının hazırlanmış olması gerekiyor.

webapp dizini altında **i18n** klasörü

içinde `i18n.properties` `i18n_tr.properties` `i18n_en.properties` `i18n_de.properties` şeklinde dil dosyalarınızı hazırlayın.

Örnek:

`i18n.properties`

`welcomeText=Bienvenido!`

`i18n_tr.properties`

`welcomeText=Hoş geldiniz!`

`i18n_en.properties`

`welcomeText=Welcome!`

`i18n_de.properties`

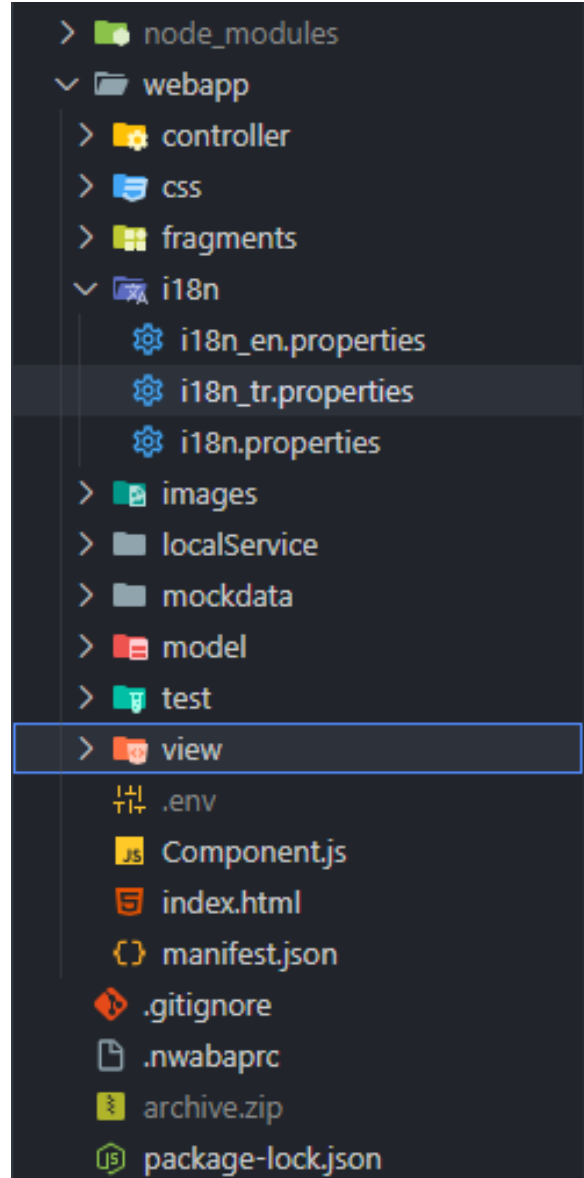
`welcomeText=Herzlich Willkommen!!`

`i18n.properties` **dosyası** İspanyolca diline ait verileri içeriyor.

`i18n_tr.properties` dosyası Türkçe,

`i18n_en.properties` dosyası İngilizce,

`i18n_de.properties` dosyası Almanca içeriyor.



Uygulama ilk çalıştırıldığında `manifest.json` dosyasında belirttiğiniz i18n dilde çalışır. Mesela aşağıdaki örnekte belirlenen yapıya göre uygulama açıldığında default olarak İspanyolca dilinde açılacak. Eğer uygulama ilk açıldığında Almanca

dilinde açılсын istiyorsan `"i18n": "i18n/i18n_de.properties"`, şeklinde dil seçimimizi belirtmeliyiz.

```
"sap.app": {
  "id": "test",
  "i18n": "i18n/i18n.properties",
  "applicationVersion": {
    "version": "0.0.1"
  },
  ...
}
```

Dil dosyasının oluşturulması ve default dil seçimi belirlenmesinin ardından isterseniz kullanıcıların da dil seçimi yapmasını sağlayabilirsiniz. Aşağıdaki kodlar ile bu işlem yapılabilir.

View.controller.js

```
// onInit içinde id'ye göre seçip default dil ataması yapılabilir.
onInit: function () {
  var oComboBox = this.byId("languageComboBox");
  oComboBox.setSelectedKey("tr"); // Varsayılan olarak Türkçe'yi seç
},

// tam olarak dil seçimine göre i18n dosyasını seçen alan.
onLanguageChange: function (oEvent) {
  var sSelectedLanguage = oEvent.getSource().getSelectedKey(); // Dil seçim
  ini al
  var oResourceModel = new sap.ui.model.resource.ResourceModel({
    bundleName: "test.i18n.i18n",
    bundleLocale: sSelectedLanguage
  });
}
```

```
});
this.getView().setModel(oResourceModel, "i18n");
},
```

View.view.xml

```
<!-- Dil Seçimi - Örnek sayfa kodudur → View.view.xml dosya içeriği→
<mvc:View
  xmlns:mvc="sap.ui.core.mvc"
  xmlns="sap.m"
  xmlns:core="sap.ui.core"
  controllerName="test.controller.Main">
  <Page title="{i18n>appTitle}">
    <content>
      <OverflowToolbar>
        <content>
          <ComboBox id="languageComboBox" selectionChange=".onLanguageChange" selectedKey="tr">
            <items>
              <core:Item key="tr" text="Türkçe" />
              <core:Item key="en" text="English" />
              <core:Item key="de" text="Deutsch" />
            </items>
          </ComboBox>
        </content>
      </OverflowToolbar>
      <Text text="{i18n>welcomeText}" />
      <Button text="{i18n>buttonSubmit}" press=".onButtonPress" />
    </content>
  </Page>
</mvc:View>
```

```
<!-- sadece aşağıdaki bölüm Dil Seçimi ComboBox için yeterlidir. →
<m:ComboBox id="languageComboBox" selectionChange=".onLanguageChange" selectedKey="tr">
  <core:Item key="tr" text="Türkçe" />
  <core:Item key="en" text="English" />
  <core:Item key="de" text="Deutsch" />
</m:ComboBox>
```

manifest.json

```
"models": {
  "i18n": {
    "type": "sap.ui.model.resource.ResourceModel",
    "settings": {
      "bundleName": "yourProjectNamespace.i18n.i18n"
    }
  }
}
```

Dinamik olarak dil seçimi yapmak

Default dil belirleme, kullanıcının dili seçmesinin ardından; dil seçiminde, kullanıcının tarayıcı diline göre veya başka bir parametreye göre dinamik bir dil seçimi yapmak isterseniz, şu şekilde bir kontrol yazabilirsiniz:

Aşağıdaki örneği 1 tane controller'a eklemek için kullanabilirsiniz.

```
onInit: function () {
  var oLanguage = sap.ui.getCore().getConfiguration().getLanguage();
  if (oLanguage === "es") {
    this._setLanguage("es"); // İspanyolca
  } else if (oLanguage === "tr") {
    this._setLanguage("tr"); // Türkçe
  }
}
```

```
} else {  
    this._setLanguage("en"); // Varsayılan olarak İngilizce  
}  
}
```

2 senaryomuz olsun;

1. Hali hazırda bir uygulamam var zaten ve buna çoklu dil seçeneği ekleyeceğim.
2. Yeni bir proje yapıyorum ve buna çoklu dil seçeneği ekleyeceğim.

Hali hazırda var olan projelere çoklu dil seçeneği eklemek

Evet, mevcut projelerde BaseController kullanmak bazen zor olabilir, çünkü mevcut controller'lar zaten belirli işlevlere sahip ve bunlara sonradan müdahale etmek bazen karmaşık olabilir. Ancak yine de merkezi bir dil yönetimi sağlamak için başka bir yaklaşım kullanabiliriz.

Alternatif Çözüm: Controller'larda Tekrarlanan Kodu Merkezi Bir Yere Taşıma

Mevcut projede BaseController kullanmak zor oluyorsa, aşağıdaki gibi merkezi bir **modül** (utility) veya **global bir fonksiyon** oluşturabilirsiniz. Bu, dil yönetimi işlevini merkezi bir yerde tutarak her controller'da sadece bir fonksiyon çağırarak uygulamanın dilini ayarlamanızı sağlar.

1. Dil Yönetim Fonksiyonu (Utility Module)

Dil yönetimi için merkezi bir modül oluşturabilirsiniz. Bu modül, uygulamanın dilini ayarlayan fonksiyonları içerir. Böylece, bu fonksiyonu her controller'da çağırarak dil ayarlarını yönetebilirsiniz.

LanguageManager.js (Dil Yönetim Modülü):

```
javascript  
Kodu kopyala  
sap.ui.define([], function () {
```

```

"use strict";

return {

    // Dil ayarlarını yapacak fonksiyon
    setLanguage: function (oController) {
        var oLanguage = sap.ui.getCore().getConfiguration().getLanguage();
        if (oLanguage === "es") {
            this._applyLanguage(oController, "es"); // İspanyolca
        } else if (oLanguage === "tr") {
            this._applyLanguage(oController, "tr"); // Türkçe
        } else {
            this._applyLanguage(oController, "en"); // Varsayılan olarak İngilizce
        }
    },

    // Dil ayarlarını modelle uygulayacak fonksiyon
    _applyLanguage: function (oController, sLanguage) {
        var oResourceModel = new sap.ui.model.resource.ResourceModel({
            bundleName: "test.i18n.i18n",
            bundleLocale: sLanguage
        });
        oController.getView().setModel(oResourceModel, "i18n");
    }
};
});

```

2. Controller'larda Dil Yönetimini Kullanmak

Bu modülü her controller'da sadece bir kez çağırarak dil ayarlarını uygulayabilirsiniz. Her controller'da `onInit` fonksiyonunu yazarken, dil yönetim fonksiyonunu çağırmanız yeterli olur.

MainController.js (Her Controller):

```

javascript
Kodu kopyala
sap.ui.define([
    "test/util/LanguageManager" // Dil yönetim modülünü içeri aktar
], function (LanguageManager) {
    "use strict";

    return sap.ui.controller("test.controller.Main", {

        onInit: function () {
            // Dil ayarlarını uygulamak için modülü kullan
            LanguageManager.setLanguage(this);
        },

        // Diğer controller fonksiyonları...
    });
});

```

3. Avantajlar:

- **Kod Tekrarını Azaltır:** Her controller'da dil ayarlarını yazmak yerine, merkezi bir fonksiyon çağırırsınız.
- **Kolay Bakım:** `LanguageManager` modülünde değişiklik yaparak tüm controller'ları etkileyebilirsiniz.
- **Esneklik:** Dil yönetimini merkezi bir yerde tutarak, gerektiğinde dil değiştirme işlemlerini kolayca yönetebilirsiniz.

4. Ekstra Özellikler:

Eğer dil seçimini kullanıcıya sunmak isterseniz, `LanguageManager` modülünü geliştirerek, kullanıcı dilini değiştirdiğinde dinamik olarak güncellenmesini sağlayabilirsiniz. Örneğin, bir dil değişim fonksiyonu ekleyebilirsiniz.

Dil Değiştirme Fonksiyonu:

javascript

Kodu kopyala

```
setLanguage: function (oController) {
    var oLanguage = sap.ui.getCore().getConfiguration().getLanguage();
    if (oLanguage === "es") {
        this._applyLanguage(oController, "es"); // İspanyolca
    } else if (oLanguage === "tr") {
        this._applyLanguage(oController, "tr"); // Türkçe
    } else {
        this._applyLanguage(oController, "en"); // Varsayılan olarak İngilizce
    }
},

changeLanguage: function (oController, sLanguage) {
    // Kullanıcı dilini değiştirdiğinde güncelleme yap
    sap.ui.getCore().getConfiguration().setLanguage(sLanguage);
    this._applyLanguage(oController, sLanguage);
}
```

Bu fonksiyonu, dil seçimi ile ilgili bir `ComboBox` kullanarak çağırabilirsiniz.

Sonuç:

Mevcut projelerde BaseController kullanmak yerine, dil yönetimi için merkezi bir **modül** (utility) oluşturmak hem esnek hem de uygulamanın her yerinde kolayca kullanılabilir bir çözüm sunar. Böylece her controller'da aynı dil ayarlarını tekrar yazmak yerine, sadece bir fonksiyon çağırarak dil yönetimini sağlayabilirsiniz.

Yeni projede çoklu dil seçeneği eklemek

merkezi bir yerden kontrol etmek için ortak bir üst sınıf (base controller) kullanabilirsiniz. SAPUI5'in MVC yapısında, tüm controller'larınızı tek bir üst

sınıftan türetirseniz, her controller'da aynı kodu tekrar yazmanıza gerek kalmaz.

1. Bir Base Controller (Temel Controller) Oluşturun:

Öncelikle, ortak işlevselliği sağlayacak bir base controller (temel controller) oluşturun. Bu controller, dil ayarlarını tüm alt controller'lar için merkezi bir yerden yönetecektir.

BaseController.js (Temel Controller):

```
javascript
Kodu kopyala
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/model/resource/ResourceModel"
], function (Controller, ResourceModel) {
    "use strict";

    return Controller.extend("test.controller.BaseController", {

        onInit: function () {
            var oLanguage = sap.ui.getCore().getConfiguration().getLanguage();
            if (oLanguage === "es") {
                this._setLanguage("es"); // İspanyolca
            } else if (oLanguage === "tr") {
                this._setLanguage("tr"); // Türkçe
            } else {
                this._setLanguage("en"); // Varsayılan olarak İngilizce
            }
        },

        _setLanguage: function (sLanguage) {
            var oResourceModel = new ResourceModel({
                bundleName: "test.i18n.i18n",
                bundleLocale: sLanguage
            });
            this.getView().setModel(oResourceModel, "i18n");
        }
    });
});
```

```
}  
});  
});
```

2. Controller'larınızı BaseController'dan Türetin:

Artık her bir controller, dil yönetimi işlevselliğini BaseController'dan alacak şekilde şu şekilde düzenlenebilir:

Controller.js (Her Controller'da):

```
javascript  
Kodu kopyala  
sap.ui.define([  
    "test/controller/BaseController"  
], function (BaseController) {  
    "use strict";  
  
    return BaseController.extend("test.controller.Main", {  
        // Main controller'a özgü fonksiyonlar burada yer alabilir  
    });  
});
```

Yukarıdaki gibi her controller'ınızda sadece `BaseController` 'ı türetmeniz yeterli olacak. Bu şekilde, `onInit` ve `_setLanguage` işlevselliği her controller'a otomatik olarak gelir.

3. BaseController Kullanımının Avantajları:

- **Tekrarlamayı Önler:** Her controller'da dil ayarlarını tekrar yazmanıza gerek kalmaz.
- **Merkezi Yönetim:** Dil seçimi ve diğer ortak işlevsellikleri merkezi bir yerden yönetebilirsiniz.
- **Kolay Bakım:** Bir değişiklik yapmanız gerektiğinde, sadece `BaseController` üzerinde değişiklik yaparak tüm controller'larınızı etkileyebilirsiniz.

4. Ekstra Özellik:

Eğer `onInit` fonksiyonuna daha fazla işlevsellik eklemek isterseniz, örneğin belirli bir controller için özel dil ayarları, bu işlevselliği BaseController içinde tanımlayıp, her controller'a ek olarak özelleştirebilirsiniz.

Sonuç:

Bu şekilde, her controller'ınızda aynı `onInit` fonksiyonunu tekrarlamak yerine, sadece bir kez `BaseController` kullanarak dil ayarlarını tüm controller'lara uygulayabilirsiniz. Bu yaklaşım, kodunuzu daha düzenli ve sürdürülebilir hale getirecektir.